

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ**

федеральное государственное автономное образовательное
учреждение высшего образования



**«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
ТОМСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»**

Инженерная школа информационных технологий и робототехники

09.04.04 «Программная инженерия»

Отделение информационных технологий

Отчет по лабораторной работе №2

по дисциплине «Системы параллельной обработки данных»

Выполнил:

Студент группы 8МП5Л

_____ Шаврин А.П.

Проверил:

К.т.н., доцент ОИТ

_____ Аксенов С.В.

Цель

Получить навыки создания настройки стратегии движения агентами в многоагентной среде с неопределенностью с применением многопроцессных вычислений.

Задание

Создать многоагентную среду, в которой имеются несколько агентов-доставщиков полезного груза по городу. Движение каждого агента-доставщика осуществляется одним процессом. Движение других транспортных средств (ботов) контролируется другим процессом. Опционально: можно сделать для каждого бота свой процесс.

Для этого необходимо сформировать карту, в которой будет перемещаться агент и другие транспортные средства. Пример карты для одного агента в файле “citymap.txt”. Минимальные размеры карты 30 x 30.

Пояснения к карте:

0 (а также 2, 3, 4) – области на карте, куда агент может попасть (дорога).

1 – на карте отмечены препятствия, куда агент не может попасть (стены).

2 – место, с которого агент начинает движение по карте.

3 – места, с которых с определенной вероятностью выходят другие транспортные средства (вероятность этого мала, например 0.02).

4 – область, в которую должен попасть агент.

В задаче попробуем также посмотреть, как будет решаться доставка груза, если доставщиков будет 2 и 3. Т.е. для каждого из доставщиков нужно установить место, из которого он будет выходить и куда ему необходимо доставить груз.

На каждой итерации:

Агент делает одно из действий: влево, вправо, вверх, вниз или остановка. Транспортные средства тоже делают одно из этих движений. Для каждого транспортного средства (бота) генерируется движение в область, куда можно попасть (0, 2, 3, 4) при этом, если транспортное средство начинает движение из тупика, то выбирается единственное направление – выход из тупика), при достижении перекрестка, применяется следующее условие: с вероятностью 10% бот делает остановку, а на следующий ход бот разворачивается и движется в обратном направлении, оставшиеся 90% равномерно распределяются между другими вариантами движения через перекресток, например если бот доехал до перекрестка с нижней клетки и есть варианты двигаться вверх, налево и направо, то с вероятностью 10% он делает остановку, а затем перемещается вниз, с вероятностью 90% становится в центр перекрестка, а затем выбирает движение вверх с вероятностью $1/3$, направо с вероятностью $1/3$ или налево с вероятностью $1/3$. Если на развилке только два возможных пути, то вероятности выборка каждого $1/2$. Если бот достигает тупика, то он разворачивается и едет в противоположную сторону.

В случае столкновения ботов между собой или агента с ботом или агентов между собой, происходит авария и транспортные средства удаляются. Если столкнулся агент, то мы признаем эпизод движения агента неуспешным. При начальном выходе из области 3 для транспортного средства (бота) генерируется число количество шагов (случайная величина от 15 до 150). В течение этого числа итераций транспортное средство двигается, а по достижению этого числа убирается из карты (припарковывается или заезжает в гараж).

Для каждого агента доставщика нужно тоже задать стратегию движения по городу. Выполнить оптимизацию стратегий каждого из агентов. Будем считать, что агент «видит» на две клетки дороги от себя во всех направлениях, а также знает координаты своего положения на карте.

Исследовать как будет изменяться среднее количество шагов, за которое агент доставит груз, а также вероятность успеха в достижении точки доставки в зависимости от скорости генерации других транспортных средств и стратегии движения первоначальной и после оптимизации с применением алгоритмов SARSA и Q-обучения.

Посмотреть, насколько меняются конечные стратегии в зависимости от количества доставщиков.

Выполнение работы

Для проведения экспериментов была составлена карта города размером 30×30 клеток и сохранена в файл citymap.txt (Рисунок 1). Визуализация сгенерированной карты представлена на рисунке 2. Зеленые - точки спавна агента, красный - агент, желтые - точки спавна ботов, синие — боты, оранжевая — точка доставки.

[illegible]

Рисунок 1. Карта города - в файле citymap.txt.

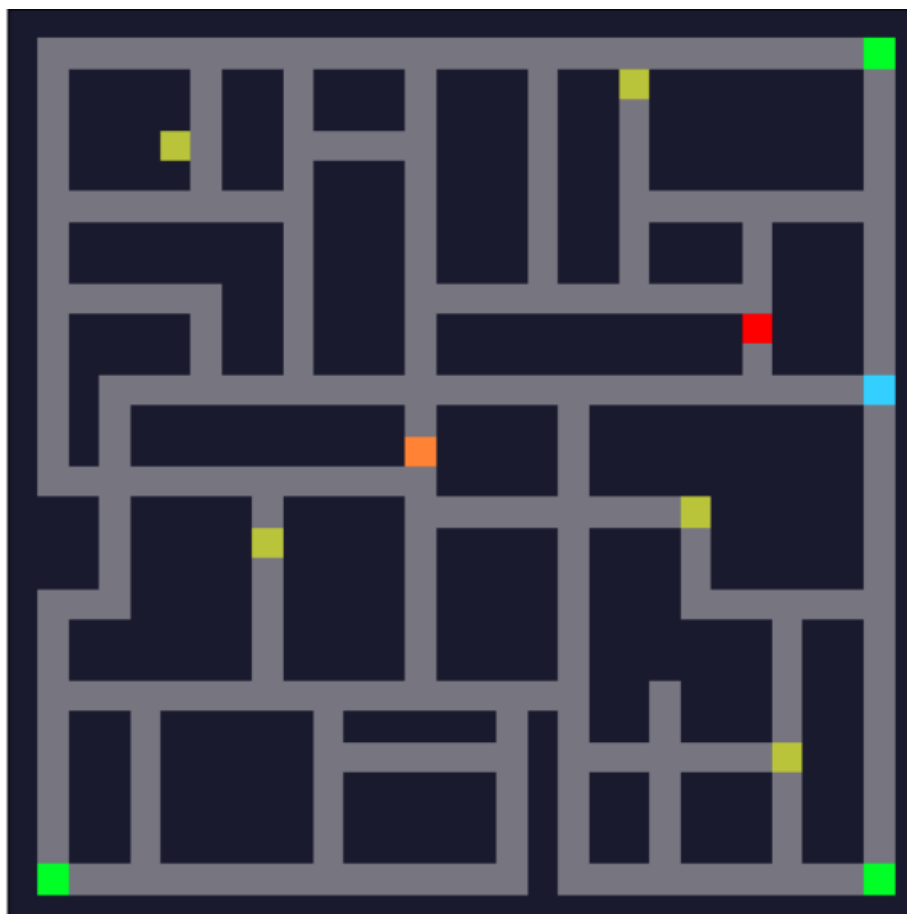


Рисунок 2. Карта города - визуализация.

Сначала был реализован модуль **map_utils.py** для загрузки карты. Он обеспечивает загрузку карты из текстового файла и автоматическое извлечение: Стартовых позиций (значение 2), Целей (значение 4), Спавн-поинтов ботов (значение 3)

Далее был реализован модуль **agent.py**, реализующий логику движения агентов. Ключевые особенности:

- Поддержка 3х типов стратегий: **bot_like**: детерминированная стратегия, имитирующая движение ботов (разворот в тупиках, случайная остановка на перекрестках).
- **SARSA** и **Q-learning**: алгоритмы обучения с подкреплением с различными подходами к обновлению Q-значений.

Функция **get_state()** формирует дискретное состояние агента, включающее координаты агента, наличие ботов в радиусе 2 клеток по каждому направлению и проходимость клеток в радиусе 2 клеток. Направление к цели (**dx**, **dy**) –

важное улучшение, позволяющее агенту ориентироваться без знания всей карты.

Функция `choose_action_q()` реализует ϵ -greedy политику выбора действия.

Процессная функция `agent_process()` работает в бесконечном цикле, принимая команды от основного процесса через Pipe:

- `reset` – сброс агента в стартовую позицию
- `step` – выполнение одного шага с обновлением Q-таблицы (для RL-стратегий)
- `stop` – завершение работы процесса

После был написан модуль **bots.py** для управления ботами.

Функция `free_neighbors()` возвращает список доступных для перемещения соседних клеток, а процессная функция `bot_process()`:

- Принимает команду `step` с текущим списком ботов
- Генерирует новых ботов в спавн-поинтах (клетки 3) с вероятностью `p_gen`
- Каждый бот имеет случайное время жизни от 15 до 150 шагов
- Логика движения бота аналогична `bot_like` стратегии агента: В тупике – разворот, на перекрестке с 4 направлениями – с вероятностью 10% остановка с последующим разворотом, в остальных случаях – случайный выбор из доступных направлений

Затем реализован модуль `simulator.py` для многопроцессной симуляция среды. Это ключевой модуль, координирующий взаимодействие всех компонентов. Основные методы:

- `start_agent()` – создает отдельные процессы для каждого агента и процесс для ботов
- `reset_agents()` – отправляет команду сброса всем агентам
- `run_episode()` – основной цикл симуляции одного эпизода

Особенности обработки взаимодействий:

- Столкновения бот-бот: боты, оказавшиеся на одной клетке, удаляются (имитация аварии).

- Столкновения агент-бот: если агент занимает клетку с ботом – эпизод признается неуспешным.
- Столкновения агент-агент: если два активных агента оказываются на одной клетке – авария.

Взаимное избегание: при планировании хода каждый агент "видит" других агентов как препятствия.

После был написан модуль **result_utils.py** для сохранения результатов. Модуль реализует функции:

- `save_results()` – сериализация результатов в pickle-файл
- `load_results()` – загрузка результатов
- `load_all_results()` – загрузка результатов для всех количеств агентов

Это позволяет проводить эксперименты и построение графиков отдельно, без повторного запуска длительного обучения.

Затем реализован модуль **visualization.py** для визуализации и анализа результатов.

- `draw_frame()` – отрисовка одного кадра с использованием цветовой карты
- `create_gif_from_history()` – создание GIF-анимации напрямую из истории эпизода
- `visualize_experiment()` – полный цикл: обучение (если нужно для алгоритмов с обучением), запуск эпизода и сохранение GIF
- `plot_comparison_graphs()` – построение сравнительных графиков для всех количеств агентов

Цветовая карта:

#1a1a2e – стены (темно-серый)

#777580 – дороги (светло-серый)

#ff0000 – агенты (красный)

#32cfff – боты (бирюзовый)

#ff8235 – цель (оранжевый)

#bac43a – спавн-поинты ботов (желтый)

#00ff26 — спавн-поинты агентов (зеленый)

Файл `main.py` является точкой входа в программу и объединяет все компоненты. Для каждого количества агентов (1, 2, 3) и каждой стратегии:

- Запускается обучение (для RL-стратегий):
- Проводится оценка на 200 эпизодах
- Результаты сохраняются в `saved_results/`

Результаты экспериментов

После завершения серии экспериментов были получены статистические данные, сохраненные в файл `experiment_results_rl.csv`. На основе этих данных построены графики, позволяющие оценить эффективность различных стратегий поведения агента. Эксперименты проводились при трех значениях вероятности генерации ботов: 0.01, 0.03, 0.05. Для каждого значения оценивались: Среднее количество шагов до достижения цели (только успешные эпизоды) и вероятность успешного достижения цели.

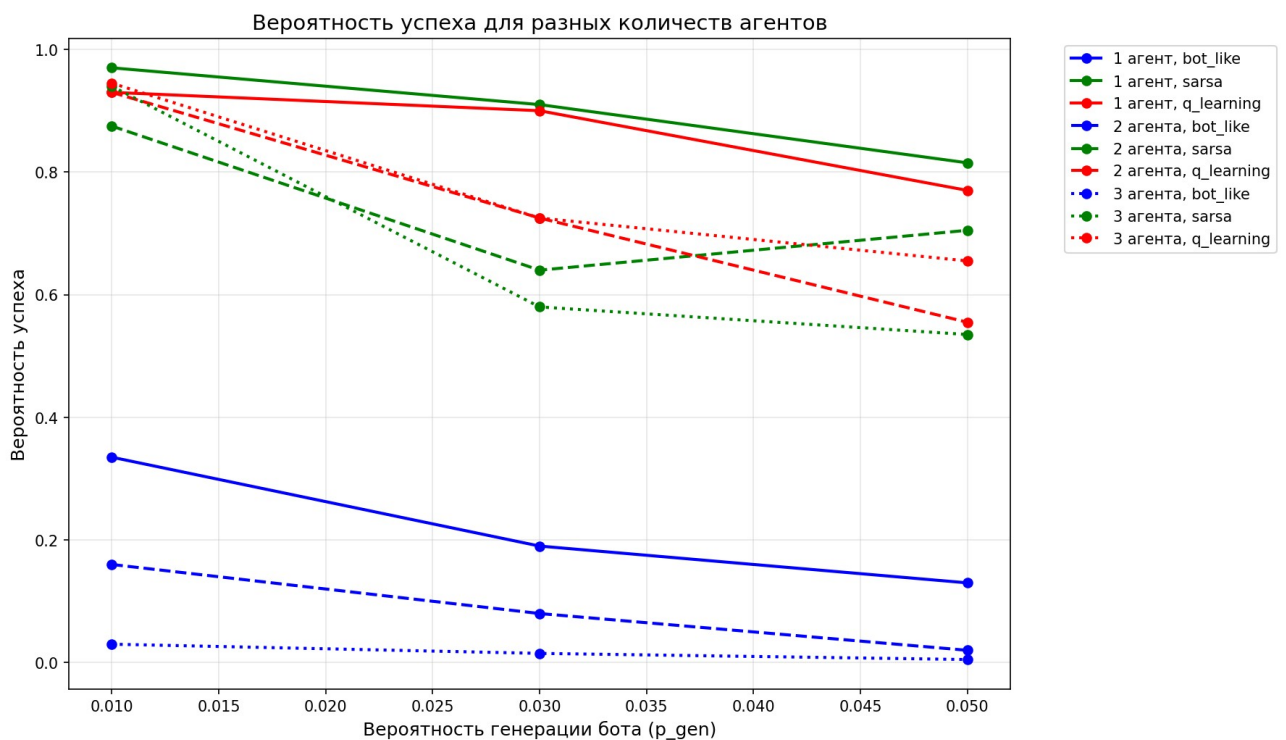


Рисунок 1. Вероятность успеха.

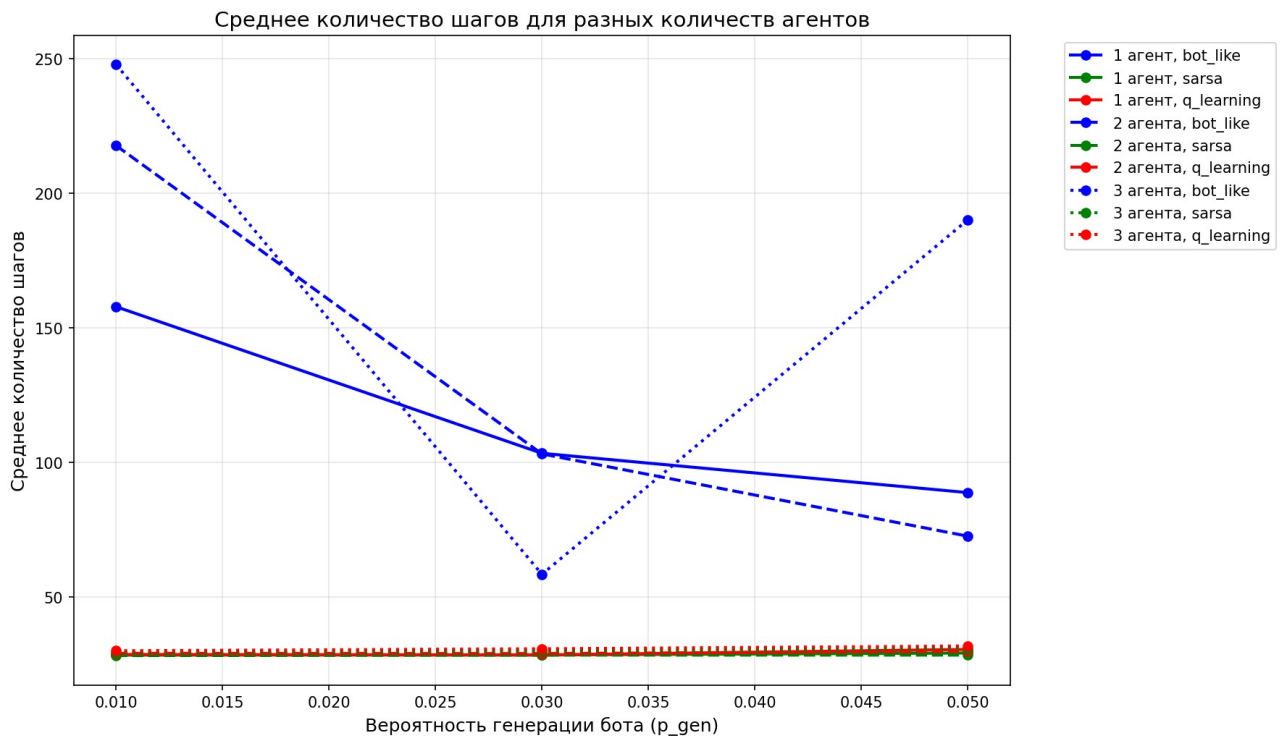


Рисунок 2. Среднее число шагов.

Анализ результатов

1. Сравнение стратегий

- Детерминированная стратегия `bot_like` показывает наихудшие результаты: вероятность успеха при 3 агентах и $p_{gen}=0.05$ падает до 0.005 (0.5%), среднее количество шагов сильно колеблется и не имеет стабильной тенденции, стратегия не обучается и не адаптируется к присутствию других агентов и ботов.
- Алгоритмы обучения с подкреплением демонстрируют значительное преимущество: вероятность успеха для 3 агентов при $p_{gen}=0.01$ достигает 0.945 (Q-learning), среднее количество шагов стабильно находится в диапазоне 28-32, что близко к оптимальному пути.
- SARSA и Q-learning показывают сопоставимые результаты, хотя Q-learning имеет небольшое преимущество при высоком уровне шума

2. Влияние количества агентов

- 1 агент: наилучшие показатели, вероятность успеха SARSA при $p_{gen}=0.01$ – 0.97
- 2 агента: некоторое снижение эффективности, особенно для SARSA при $p_{gen}=0.05$ (0.705 против 0.815 у 1 агента)
- 3 агента: при низкой вероятности генерации ботов (0.01) показатели сопоставимы с 1 агентом, а при высокой вероятности (0.05) наблюдается значительное снижение: SARSA с 0.815 до 0.535, также наблюдается увеличение среднего количества шагов с ~28 до ~31 шага

3. Влияние вероятности генерации ботов

Для RL-стратегий наблюдается закономерное снижение вероятности успеха при увеличении p_{gen} :

- 1 агент (Q-learning): $0.93 \rightarrow 0.90 \rightarrow 0.77$
- 2 агента (Q-learning): $0.93 \rightarrow 0.725 \rightarrow 0.555$
- 3 агента (Q-learning): $0.945 \rightarrow 0.725 \rightarrow 0.655$

При этом среднее количество шагов в успешных эпизодах остается практически неизменным – агенты, которые достигают цели, делают это за оптимальное число шагов, несмотря на помехи.

Q-learning показывает более высокую вероятность успеха при высоких p_{gen} (0.655 против 0.535 для 3 агентов при $p_{gen}=0.05$), что объясняется его off-policy природой – он стремится к оптимальной стратегии, даже если она более рискованна. SARSA, напротив, более консервативен, так как учитывает реальные действия агента.

Вывод

В результате выполнения лабораторной работы была разработана и реализована многоагентная система доставки грузов в городской среде с неопределенностью. Создана карта размером 30×30 клеток с тремя стартовыми позициями агентов, общей точкой доставки и спавн-поинтами для ботов. Реализована многопроцессная архитектура, где каждый агент управляется отдельным процессом, а боты – одним процессом. Для агентов были реализованы три стратегии движения: детерминированная *bot_like*, SARSA и Q-learning. Проведены эксперименты для 1, 2 и 3 агентов при вероятностях генерации ботов 0.01, 0.03 и 0.05. В результате установлено, что детерминированная стратегия *bot_like* показала крайне низкую эффективность – при трех агентах и $p_{gen}=0.05$ вероятность успеха составила всего 0.005. Алгоритмы обучения с подкреплением продемонстрировали высокую эффективность: вероятность успеха для трех агентов при $p_{gen}=0.01$ достигла 0.945 (Q-learning), а среднее количество шагов стабильно находилось в диапазоне 28–32, что близко к оптимальному пути. Увеличение числа агентов и рост вероятности генерации ботов закономерно снижают вероятность успеха, однако среднее количество шагов в успешных эпизодах остается неизменным. Q-learning показал более высокую устойчивость при высоком уровне помех по сравнению с SARSA. Таким образом, цель работы достигнута, экспериментально подтверждено превосходство алгоритмов обучения с подкреплением над детерминированными стратегиями в многоагентной среде с неопределенностью.

ИСХОДНЫЙ КОД ПРОГРАММЫ

[illegible]

```
import numpy as np
```

14

```

        elif val == 4:
            goals.append((x, y))
        elif val == 3:
            spawn_points.append((x, y))

    if len(starts) < 1 or len(goals) < 1:
        raise ValueError("Карта должна содержать хотя бы один старт (2) и одну цель (4).")

    return grid, (height, width), starts, goals, spawn_points

```

result_utils.py

```

import os
import pickle

def save_results(results, filename):
    """Сохраняет результаты в файл"""
    os.makedirs('saved_results', exist_ok=True)
    with open(os.path.join('saved_results', filename), 'wb') as f:
        pickle.dump(results, f)
    print(f"Результаты сохранены: saved_results/{filename}")

def load_results(filename):
    """Загружает результаты из файла"""
    filepath = os.path.join('saved_results', filename)
    if os.path.exists(filepath):
        with open(filepath, 'rb') as f:
            results = pickle.load(f)
        print(f"Результаты загружены: saved_results/{filename}")
        return results
    return None

def load_all_results():
    """Загружает все доступные сохраненные результаты"""
    results_dict = {}

    # Попробуем загрузить результаты для разных количеств агентов
    for name, filename in [('1 агент', 'results_1agent.pkl'),
                           ('2 агента', 'results_2agent.pkl'),
                           ('3 агента', 'results_3agent.pkl')]:
        data = load_results(filename)
        if data is not None:
            results_dict[name] = data
            print(f"Загружены результаты: {name}")
        else:
            print(f"Загружены результаты: {name} {filename}", data)

    return results_dict

```

agent.py

```

import random
import numpy as np
from collections import defaultdict

DIRECTIONS = [(1, 0), (-1, 0), (0, 1), (0, -1)]
ACTIONS = [(1,0), (-1,0), (0,1), (0,-1), (0,0)]

def free_neighbors(pos, static_grid):

```

```

x, y = pos
h, w = static_grid.shape
neighbors = []
for dx, dy in DIRECTIONS:
    nx, ny = x + dx, y + dy
    if 0 <= nx < w and 0 <= ny < h and static_grid[ny, nx]:
        neighbors.append((nx, ny))
return neighbors

def get_valid_actions(pos, grid):
    valid = []
    for dx, dy in ACTIONS[:-1]:
        nx, ny = pos[0]+dx, pos[1]+dy
        if 0 <= nx < grid.shape[1] and 0 <= ny < grid.shape[0] and grid[ny, nx]:
            valid.append((dx, dy))
    valid.append((0,0))
    return valid

def get_state(pos, bot_positions, static_grid, goal=None):
    x, y = pos
    h, w = static_grid.shape
    bot_set = set(bot_positions) if bot_positions else set()

    def obstacles_in_dir(dx, dy):
        res = []
        for d in (1, 2):
            nx, ny = x + dx*d, y + dy*d
            if 0 <= nx < w and 0 <= ny < h:
                res.append((nx, ny) in bot_set)
            else:
                res.append(False)
        return res

    up_obs = obstacles_in_dir(0, -1)
    down_obs = obstacles_in_dir(0, 1)
    left_obs = obstacles_in_dir(-1, 0)
    right_obs = obstacles_in_dir(1, 0)

    def is_passable(nx, ny):
        return 0 <= nx < w and 0 <= ny < h and static_grid[ny, nx]

    up_pass = [is_passable(x, y-1), is_passable(x, y-2)]
    down_pass = [is_passable(x, y+1), is_passable(x, y+2)]
    left_pass = [is_passable(x-1, y), is_passable(x-2, y)]
    right_pass = [is_passable(x+1, y), is_passable(x+2, y)]

    if goal:
        goal_dx = np.sign(goal[0] - x)
        goal_dy = np.sign(goal[1] - y)
    else:
        goal_dx, goal_dy = 0, 0

    return (x, y,
            up_obs[0], up_obs[1], down_obs[0], down_obs[1],
            left_obs[0], left_obs[1], right_obs[0], right_obs[1],
            up_pass[0], up_pass[1], down_pass[0], down_pass[1],
            left_pass[0], left_pass[1], right_pass[0], right_pass[1],
            goal_dx, goal_dy)

```

```

def choose_action_q(Q, state, epsilon, rng, valid_actions):
    if rng.random() < epsilon:
        return rng.choice(valid_actions)
    if state not in Q:
        Q[state] = {a: 0.0 for a in ACTIONS}
    q_vals = {a: Q[state][a] for a in valid_actions}
    max_q = max(q_vals.values())
    max_actions = [a for a, v in q_vals.items() if v == max_q]
    return rng.choice(max_actions)

def bot_like_move(pos, prev, stopped, static_grid, rng):
    """Стратегия движения, идентичная обычным ботам."""
    if pos is None:
        return pos, prev, stopped

    if stopped:
        next_pos = prev if prev is not None else pos
        new_stopped = False
        new_prev = pos
        return next_pos, new_prev, new_stopped

    all_nb = free_neighbors(pos, static_grid)
    candidates = [n for n in all_nb if n != prev] if prev is not None else all_nb
    if not candidates:
        candidates = [prev] if prev is not None else all_nb
    if not candidates:
        return pos, pos, False

    total_passable = len(all_nb)
    if total_passable == 1:
        chosen = candidates[0]
    elif total_passable == 2:
        chosen = candidates[0]
    elif total_passable == 3:
        chosen = rng.choice(candidates)
    elif total_passable == 4:
        if prev is not None and rng.random() < 0.1:
            return pos, prev, True
        else:
            chosen = rng.choice(candidates)
    else:
        chosen = candidates[0]

    return chosen, pos, False

def agent_process(conn, static_grid, goal, strategy, rng_seed=None):
    rng = random.Random(rng_seed)
    agent_pos = None
    prev = None
    stopped = False
    last_action = None
    alpha = 0.5
    gamma = 0.9
    epsilon = 0.1
    Q = None

    if strategy in ('sarsa', 'q_learning'):
        Q = defaultdict(lambda: {a: 0.0 for a in ACTIONS})

```

```

try:
    while True:
        msg = conn.recv()
        if msg == 'stop':
            break
        elif msg[0] == 'reset':
            agent_pos, eps = msg[1], msg[2]
            prev = None
            stopped = False
            last_action = None
            if strategy in ('sarsa', 'q_learning'):
                epsilon = eps
        elif msg[0] == 'step':
            bot_positions = msg[1]
            if strategy == 'bot_like':
                if agent_pos is None:
                    conn.send((agent_pos, prev, stopped))
                else:
                    new_pos, new_prev, new_stopped = bot_like_move(agent_pos, prev, stopped, static_grid, rng)
                    agent_pos, prev, stopped = new_pos, new_prev, new_stopped
                    conn.send((new_pos, new_prev, new_stopped))
            elif strategy in ('sarsa', 'q_learning'):
                state = get_state(agent_pos, bot_positions, static_grid, goal)
                valid_actions = get_valid_actions(agent_pos, static_grid)

                if last_action is None:
                    action = choose_action_q(Q, state, epsilon, rng, valid_actions)
                else:
                    action = last_action

                nx, ny = agent_pos[0] + action[0], agent_pos[1] + action[1]
                if 0 <= nx < static_grid.shape[1] and 0 <= ny < static_grid.shape[0] and static_grid[ny, nx]:
                    new_pos = (nx, ny)
                else:
                    new_pos = agent_pos

                reward = -1.0
                if new_pos == goal:
                    reward = 100.0
                elif new_pos == agent_pos:
                    reward = -2.0

                next_state = get_state(new_pos, bot_positions, static_grid, goal)
                next_valid = get_valid_actions(new_pos, static_grid)
                next_action = choose_action_q(Q, next_state, epsilon, rng, next_valid)

                if strategy == 'sarsa':
                    Q[state][action] += alpha * (reward + gamma * Q[next_state][next_action] - Q[state][action])
                else: # Q-learning
                    max_next = max(Q[next_state][a] for a in next_valid)
                    Q[state][action] += alpha * (reward + gamma * max_next - Q[state][action])

                last_action = next_action
                agent_pos = new_pos
                prev = (agent_pos[0] - action[0], agent_pos[1] - action[1]) if action != (0, 0) else agent_pos
                conn.send((agent_pos, prev, False))
except (EOFError, BrokenPipeError, KeyboardInterrupt):
    pass
finally:

```

```

        conn.close()
bots.py
import random
import numpy as np

DIRECTIONS = [(1, 0), (-1, 0), (0, 1), (0, -1)]

def free_neighbors(pos, static_grid):
    x, y = pos
    h, w = static_grid.shape
    neighbors = []
    for dx, dy in DIRECTIONS:
        nx, ny = x + dx, y + dy
        if 0 <= nx < w and 0 <= ny < h and static_grid[ny, nx]:
            neighbors.append((nx, ny))
    return neighbors

def bot_process(conn, static_grid, spawn_points, p_gen, rng_seed=None):
    rng = random.Random(rng_seed)
    try:
        while True:
            if conn.poll(0.1):
                msg = conn.recv()
                if msg == 'stop':
                    break
                cmd, bots = msg
                occupied = set()
                for b in bots:
                    occupied.add(tuple(b['pos']))
                for sp in spawn_points:
                    if sp not in occupied and rng.random() < p_gen:
                        new_bot = {
                            'id': rng.getrandbits(31),
                            'pos': sp,
                            'prev': None,
                            'stopped': False,
                            'steps_left': rng.randint(15, 150)
                        }
                        bots.append(new_bot)
                        occupied.add(sp)

                updated_bots = []
                for b in bots:
                    steps_left = b['steps_left'] - 1
                    if steps_left <= 0:
                        continue
                    pos = b['pos']
                    prev = b['prev']
                    stopped = b['stopped']
                    new_stopped = False
                    next_pos = None

                    if stopped:
                        next_pos = prev
                        new_stopped = False
                        new_prev = pos
                    else:
                        all_nb = free_neighbors(pos, static_grid)
                        candidates = [n for n in all_nb if n != prev] if prev is not None else all_nb

```

```

if not candidates:
    candidates = [prev] if prev is not None else all_nb
if not candidates:
    next_pos = pos
    new_prev = pos
else:
    total_passable = len(all_nb)
    if total_passable == 1:
        chosen = candidates[0]
        next_pos = chosen
        new_prev = pos
    elif total_passable == 2:
        chosen = candidates[0]
        next_pos = chosen
        new_prev = pos
    elif total_passable == 3:
        chosen = rng.choice(candidates)
        next_pos = chosen
        new_prev = pos
    elif total_passable == 4:
        if prev is not None and rng.random() < 0.1:
            new_stopped = True
            next_pos = pos
            new_prev = prev
        else:
            chosen = rng.choice(candidates)
            next_pos = chosen
            new_prev = pos
    else:
        chosen = candidates[0]
        next_pos = chosen
        new_prev = pos

updated_bots.append({
    'id': b['id'],
    'pos': next_pos,
    'prev': new_prev,
    'stopped': new_stopped,
    'steps_left': steps_left
})
conn.send(updated_bots)
except (EOFError, BrokenPipeError, KeyboardInterrupt):
    pass
finally:
    conn.close()

```

simulator.py

```

from multiprocessing import Pipe, Process
from agent import agent_process
from bots import bot_process

```

class Simulator:

```

def __init__(self, grid, agent_configs, spawn_points, p_gen, max_steps=800):
    self.static_grid = (grid != 1)
    self.grid = grid
    self.agent_configs = agent_configs
    self.spawn_points = spawn_points
    self.p_gen = p_gen
    self.max_steps = max_steps

```

```

self.num_agents = len(agent_configs)
self.agent_conns = []
self.agent_procs = []

def start_agent(self):
    for cfg in self.agent_configs:
        parent_conn, child_conn = Pipe()
        p = Process(target=agent_process,
                    args=(child_conn, self.static_grid, cfg['goal'], cfg['strategy'], None))
        p.start()
        self.agent_conns.append(parent_conn)
        self.agent_procs.append(p)

def stop_agent(self):
    for conn in self.agent_conns:
        try:
            conn.send('stop')
        except:
            pass
    for p in self.agent_procs:
        p.join(timeout=2)
        if p.is_alive():
            p.terminate()
    self.agent_conns = []
    self.agent_procs = []

def reset_agents(self, epsilons=None):
    if epsilons is None:
        epsilons = [0.0] * self.num_agents
    for conn, cfg, eps in zip(self.agent_conns, self.agent_configs, epsilons):
        conn.send(('reset', cfg['start'], eps))

def run_episode(self, epsilons=None, visualize=False):
    if epsilons is None:
        epsilons = [0.0] * self.num_agents

    self.reset_agents(epsilons)

    parent_bot_conn, child_bot_conn = Pipe()
    bot_p = Process(target=bot_process,
                    args=(child_bot_conn, self.static_grid, self.spawn_points, self.p_gen, None))
    bot_p.start()

    positions = [cfg['start'] for cfg in self.agent_configs]
    prevs = [None] * self.num_agents
    stopped = [False] * self.num_agents
    active = [True] * self.num_agents
    bots = []
    history = [] if visualize else None
    all_success = False
    steps = 0
    collision_occurred = False

    while steps < self.max_steps and not collision_occurred:
        if visualize:
            history.append(([pos for pos in positions],
                           [b['pos'] for b in bots], False))

        bot_positions = [(b['pos'][0], b['pos'][1]) for b in bots]

```

```

parent_bot_conn.send(('step', bots))
bots = parent_bot_conn.recv()
bot_positions_after = [tuple(b['pos']) for b in bots]

new_positions = positions.copy()
new_prevs = prevs.copy()
new_stopped = stopped.copy()
agent_reached = [False] * self.num_agents

for i in range(self.num_agents):
    if not active[i]:
        continue

    # Все агенты (активные) считаются препятствиями
    other_agents = [positions[j] for j in range(self.num_agents) if j != i and active[j]]
    visible_obstacles = bot_positions_after + other_agents

    self.agent_conns[i].send(('step', visible_obstacles))
    new_state = self.agent_conns[i].recv()
    new_pos, new_prev, new_st = new_state

    # Проверка выхода за пределы проходимой сетки
    if not self.static_grid[new_pos[1], new_pos[0]]:
        new_pos = positions[i]
        new_prev = prevs[i]
        new_st = stopped[i]

    # Запрет на занятие клетки с другим агентом (активным)
    if new_pos != positions[i]: # если был реальный ход
        conflict = False
        for j in range(self.num_agents):
            if j != i and active[j] and new_pos == positions[j]:
                conflict = True
                break
        if conflict:
            # откатываем ход
            new_pos = positions[i]
            new_prev = prevs[i]
            new_st = stopped[i]

    new_positions[i] = new_pos
    new_prevs[i] = new_prev
    new_stopped[i] = new_st

    if tuple(new_pos) == tuple(self.agent_configs[i]['goal']):
        agent_reached[i] = True

    # Убираем столкнувшихся ботов
    pos_to_bots = {}
    for b in bots:
        p = tuple(b['pos'])
        pos_to_bots.setdefault(p, []).append(b)
    collided_ids = set()
    for p, lst in pos_to_bots.items():
        if len(lst) > 1:
            for b in lst:
                collided_ids.add(b['id'])
    if collided_ids:

```

```

bots = [b for b in bots if b['id'] not in collided_ids]
bot_positions_after = [tuple(b['pos']) for b in bots]

# Столкновения агент-бот
for i in range(self.num_agents):
    if not active[i]:
        continue
    if tuple(new_positions[i]) in bot_positions_after:
        collision_occurred = True
    if visualize:
        history.append([(new_positions[j] for j in range(self.num_agents)),
                        [b['pos'] for b in bots], True))
    break

# Столкновения агент-агент (только между активными)
if not collision_occurred:
    agent_pos_counts = {}
    for i in range(self.num_agents):
        if active[i] and not agent_reached[i]:
            p = new_positions[i]
            agent_pos_counts.setdefault(p, []).append(i)
    for p, lst in agent_pos_counts.items():
        if len(lst) > 1:
            collision_occurred = True
        if visualize:
            history.append([(new_positions[j] for j in range(self.num_agents)),
                            [b['pos'] for b in bots], True))
    break

if collision_occurred:
    all_success = False
    break

positions = new_positions
prevs = new_prevs
stopped = new_stopped

for i in range(self.num_agents):
    if agent_reached[i]:
        active[i] = False

steps += 1

if all(not a for a in active):
    all_success = True
    if visualize:
        history.append([(positions[j] for j in range(self.num_agents)),
                        [b['pos'] for b in bots], False))
    break

parent_bot_conn.send('stop')
bot_p.join(timeout=2)
if bot_p.is_alive():
    bot_p.terminate()

if visualize:
    return all_success, steps, history
return all_success, steps
visualization.py

```

```

import matplotlib.pyplot as plt
import numpy as np
import os
import imageio.v2 as imageio
from map_utils import load_map
from result_utils import load_all_results
from simulator import Simulator

AGENT_COLOR = "#ff0000"
COLLISION_COLLOR = "#000000"
BOT_COLOR = "#32cfff"
COLOR_MAP = {
    0: "#777580",
    1: "#1a1a2e",
    2: "#00ff26",
    3: "#bac43a",
    4: "#ff8235",
}

class Visualizer:
    def __init__(self):
        self.color_map = COLOR_MAP
        self.agent_color = AGENT_COLOR
        self.collision_color = COLLISION_COLLOR
        self.bot_color = BOT_COLOR

    def hex_to_rgb(self, hex_color):
        hex_color = hex_color.lstrip('#')
        return [int(hex_color[i:i+2], 16) / 255.0 for i in (0, 2, 4)]

    def draw_frame(self, step, grid, agent_positions, bot_positions, collision=False):
        """Отрисовывает кадр и возвращает изображение (без сохранения)"""
        h, w = grid.shape
        img = np.ones((h, w, 3))

        # Красим карту
        for y in range(h):
            for x in range(w):
                val = grid[y, x]
                img[y, x] = self.hex_to_rgb(self.color_map.get(val, self.color_map[0]))

        # Красим ботов
        for (bx, by) in bot_positions:
            if 0 <= by < h and 0 <= bx < w:
                img[by, bx] = self.hex_to_rgb(self.bot_color)

        # Отрисовка агентов
        for idx, (ax, ay) in enumerate(agent_positions):
            if 0 <= ay < h and 0 <= ax < w:
                if collision:
                    img[ay, ax] = self.hex_to_rgb(self.collision_color)
                else:
                    img[ay, ax] = self.hex_to_rgb(self.agent_color)

        return img

    def create_gif_from_history(self, history, grid, output_path, fps=10):
        """Создает GIF напрямую из history (без сохранения PNG)"""

```

```

if not history:
    print(f"Нет кадров")
    return None

frames = []
for agent_positions, bot_positions, collision in history:
    img = self.draw_frame(0, grid, agent_positions, bot_positions, collision)
    frames.append((img * 255).astype(np.uint8))

os.makedirs(os.path.dirname(output_path), exist_ok=True)
imageio.mimsave(output_path, frames, duration=1000/fps, loop=0)
print(f"GIF сохранен: {output_path}")
return output_path

def visualize_experiment(self, num_agents, strategy, folder_base='results'):
    """Запускает и сохраняет GIF одного запуска"""
    grid, shape, starts, goals, spawn_points = load_map('citymap.txt')
    p = 0.02

    # Создаем конфигурацию агентов
    agent_configs = []
    for i in range(num_agents):
        agent_configs.append({
            'start': starts[i % len(starts)],
            'goal': goals[0],
            'strategy': strategy
        })

    folder = os.path.join(folder_base, f'{num_agents}agents_{strategy}')
    os.makedirs(folder, exist_ok=True)

    env = Simulator(grid, agent_configs, spawn_points, p, max_steps=300)
    env.start_agent()

    # Обучаем если нужно
    if strategy in ('sarsa', 'q_learning'):
        print(f"Тренировка {num_agents} агентов ({strategy})...")
        for ep in range(300):
            eps = max(0.02, 0.3 * (1.0 - ep / 300))
            epsilons = [eps] * num_agents
            env.run_episode(epsilons=epsilons)

    # Запускаем эпизод
    epsilons = [0.0 if strategy in ('sarsa', 'q_learning') else None] * num_agents
    success, steps, history = env.run_episode(epsilons=epsilons, visualize=True)
    env.stop_agent()

    print(f" {num_agents} агент(a), {strategy}: шагов={steps}, успех={success}")

    # Создаем GIF
    gif_path = os.path.join(folder, f'{num_agents}agents_{strategy}.gif')
    self.create_gif_from_history(history, grid, gif_path, fps=10)

    return success

def plot_comparison_graphs(self, results_dict, p_vals, strategies):
    """
    Функция для построения графиков сравнения результатов
    """

```

```

os.makedirs('results', exist_ok=True)

# Настройки стилей для разных количеств агентов
colors = ['blue', 'green', 'red']
linestyles = ['-', '--', ':']

# Линейные графики для каждой метрики
for metric_name, metric_idx, ylabel in [
    ('avg_steps', 0, 'Среднее количество шагов'),
    ('success_rate', 1, 'Вероятность успеха')
]:
    plt.figure(figsize=(12, 7))

    for agents_i, (agents_label, results) in enumerate(results_dict.items()):
        for strat_i, strat in enumerate(strategies):
            if strat in results:
                vals = [results[strat][p][metric_idx] for p in p_vals]
                plt.plot(p_vals, vals,
                        color=colors[strat_i],
                        marker='o',
                        linestyle=linestyles[agents_i],
                        linewidth=2,
                        markersize=6,
                        label=f'{agents_label}, {strat}')

    plt.xlabel("Вероятность генерации бота (p_gen)", fontsize=12)
    plt.ylabel(ylabel, fontsize=12)
    plt.title(f'{ylabel} для разных количеств агентов', fontsize=14)
    plt.grid(True, alpha=0.3)
    plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left', fontsize=10)
    plt.tight_layout()
    plt.savefig(f'results/compare_{metric_name}_all.png', bbox_inches='tight', dpi=150)
    plt.close()

    print(f"Графики сохранены в 'results'")

if __name__ == "__main__":
    visualizer = Visualizer()
    # Загружаем карту для проверки стартовых точек
    grid, shape, starts, goals, spawn_points = load_map('citymap.txt')
    print(f"Доступно стартовых точек: {len(starts)}")

    # Список комбинаций: (число_агентов, стратегия)
    combinations = []

    for num_agents in [1, 2, 3]:
        if num_agents > len(starts):
            print(f"Пропускаем {num_agents} агента (нужно {num_agents} стартов, есть {len(starts)})")
            continue
        for strategy in ['bot_like', 'sarsa', 'q_learning']:
            combinations.append((num_agents, strategy))

    # Создаем gif анимации комбинаций экспериментов
    for num_agents, strategy in combinations:
        print(f"{num_agents} агент(a), стратегия {strategy}")
        visualizer.visualize_experiment(num_agents, strategy, folder_base='results')
    print("Все GIF созданы!")

    # Строим графики результатов

```

```

BOT_SPAWN_PROBS = [0.01, 0.03, 0.05]
STRATEGIES = ['bot_like', 'sarsa', 'q_learning']
results_dict = load_all_results()
print(results_dict)
if results_dict:
    print(f"Построение графиков для: {list(results_dict.keys())}")
    visualizer.plot_comparison_graphs(results_dict, BOT_SPAWN_PROBS, STRATEGIES)
else:
    print("Нет сохраненных результатов. Сначала запустите эксперименты.")

main.py
import numpy as np
from map_utils import load_map
from simulator import Simulator
import multiprocessing as mp
from result_utils import save_results

def run_experiment(p_values, agent_configs, train_episodes=300, eval_episodes=100, max_steps=1000):
    grid, shape, starts, goals, spawn_points = load_map('citymap.txt')
    results = {}
    for p in p_values:
        env = Simulator(grid, agent_configs, spawn_points, p, max_steps=max_steps)
        env.start_agent()

        strategies = [cfg['strategy'] for cfg in agent_configs]
        if any(s in ('sarsa', 'q_learning') for s in strategies):
            for ep in range(train_episodes):
                eps = max(0.02, 0.3 * (1.0 - ep / train_episodes))
                epsilons = [eps if s in ('sarsa', 'q_learning') else 0.0 for s in strategies]
                env.run_episode(epsilons=epsilons)

        successes = 0
        steps_list = []
        for _ in range(eval_episodes):
            epsilons = [0.0 if s in ('sarsa', 'q_learning') else None for s in strategies]
            success, steps = env.run_episode(epsilons=epsilons)
            if success:
                successes += 1
                steps_list.append(steps)

        env.stop_agent()

        avg_steps = np.mean(steps_list) if steps_list else float('nan')
        success_rate = successes / eval_episodes
        results[p] = (avg_steps, success_rate)
        label = f"{len(agent_configs)} агента(ов), стратегия {strategies[0]}"
        if len(set(strategies)) > 1:
            label += ' (смеш.)'
        print(f"[{label}] p={p:.3f} | success rate={success_rate:.3f} | avg steps={avg_steps:.1f}")

    return results

if __name__ == '__main__':
    mp.set_start_method('spawn', force=True)
    grid, shape, starts, goals, spawn_points = load_map('citymap.txt')

    BOT_SPAWN_PROBS = [0.01, 0.03, 0.05]
    STRATEGIES = ['bot_like', 'sarsa', 'q_learning']
    AGENTS_CONFIG = {
        0: {'start': starts[0], 'goal': goals[0]},

```

```

1: {'start': starts[1], 'goal': goals[0]},
2: {'start': starts[2], 'goal': goals[0]},
}

print("\n===== 1 АГЕНТ =====")
results_1 = {}
for strat in STRATEGIES:
    print(f"\nСтратегия: {strat}")
    cfg = [{'start': AGENTS_CONFIG[0]['start'], 'goal': AGENTS_CONFIG[0]['goal'], 'strategy': strat}]
    results_1[strat] = run_experiment(BOT_SPAWN_PROBS, cfg, train_episodes=300, eval_episodes=200,
max_steps=800)
    save_results(results_1, 'results_1agent.pkl')

print("\n===== 2 АГЕНТА =====")
results_2 = {}
for strat in STRATEGIES:
    print(f"\nСтратегия обоих агентов: {strat}")
    cfg = [
        {'start': AGENTS_CONFIG[0]['start'], 'goal': AGENTS_CONFIG[0]['goal'], 'strategy': strat},
        {'start': AGENTS_CONFIG[1]['start'], 'goal': AGENTS_CONFIG[1]['goal'], 'strategy': strat}
    ]
    # Для RL-стратегий увеличено число эпизодов и max_steps
    results_2[strat] = run_experiment(BOT_SPAWN_PROBS, cfg, train_episodes=1300, eval_episodes=200,
max_steps=1000)
    save_results(results_2, 'results_2agent.pkl')

print("\n===== 3 АГЕНТА =====")
results_3 = {}
for strat in STRATEGIES:
    print(f"\nСтратегия: {strat}")
    cfg = [
        {'start': AGENTS_CONFIG[0]['start'], 'goal': AGENTS_CONFIG[0]['goal'], 'strategy': strat},
        {'start': AGENTS_CONFIG[1]['start'], 'goal': AGENTS_CONFIG[1]['goal'], 'strategy': strat},
        {'start': AGENTS_CONFIG[2]['start'], 'goal': AGENTS_CONFIG[2]['goal'], 'strategy': strat}
    ]
    # Для RL-стратегий увеличено число эпизодов и max_steps
    results_3[strat] = run_experiment(BOT_SPAWN_PROBS, cfg, train_episodes=2000, eval_episodes=200,
max_steps=1200)
    save_results(results_3, 'results_3agent.pkl')

```